

MessageFoundry — Deployment & Network Exposure Guide

Early Access · MessageFoundry v0.3.2 · September 2026

This guide explains network binding, TLS, authentication, and access controls for each MessageFoundry channel. It documents the v0.1 native off-loopback TLS gate (Gate #4). Before exposing the engine beyond `127.0.0.1`, read the [checklist](#) and [channel matrix](#).

Design rationale for the off-loopback posture is in [adr/0002-phase2-transport-security-and-strong-auth.md](#). PHI-in-transit context is in [PHI.md](#) §4. Clustering topology is in [CLUSTERING.md](#). For the host-level antivirus exclusions and Windows Firewall rules the engine needs, see [ANTIVIRUS-FIREWALL.md](#).

On-premises by default

MessageFoundry runs **on-premises** and binds to **loopback (127.0.0.1)** by default. The API uses `[security].local_access_only = true`. Inbound listeners use `[inbound].bind_host`. With these settings, listeners cannot be reached from another host.

Loopback binding limits inbound access. Outbound traffic can still carry PHI. The engine connects to configured MLLP, REST/SOAP, FHIR, DICOMweb, SFTP/FTPS, SMTP, Direct, and database destinations. It also performs `db_lookup` and `fhir_lookup` reads, forwards syslog/SIEM logs, and sends webhook alerts. The outbound allowlists, cleartext rules, and per-connection TLS controls below also apply to loopback deployments.

The following sections explain how to bind a channel to a routable address.

Fail-closed rule (ADR 0002 §0): Startup rejects a non-loopback API bind without TLS or a trusted upstream TLS terminator. Wiring rejects off-loopback MLLP, HTTP, DICOM C-STORE SCP, and raw TCP/X12 listeners without TLS.

The cleartext-bind escapes are clamped shut on the shipped posture (ADR 0148, ADR 0092 decision 2). `serve --allow-insecure-bind` — and its config twin `[security].require_encryption_for_remote = false` — only warn-and-cross while the instance is **not** enforcing-PHI. `[security].enforcement` defaults to `enforce`. Built-in environments (`dev`, `staging`, `prod`) derive `data_class = phi`. Thus, the default instance rejects cleartext binds even with the flag. Crossing it is a deliberate, recorded loosening: set `[security].enforcement = warn`, or declare the box synthetic with `[security].handles_real_patient_data = false`. Neither is a supported production setting.

Container deployment (Docker / Kubernetes)

The engine ships as an OCI image (ADR 0017) and supports the Windows-service/NSSM installation ([SERVICE.md](#)). Containers use the same network controls.

The engine serves the optional browser console at `/ui` (ADR 0065). Neither container profile includes the `messagefoundry-webconsole` wheel. Without that wheel, the container serves only the JSON API and logs a warning. Add the wheel in a derived image to enable the console.

Refer to [../docker/README.md](#) for build and operating instructions. The scope analysis is in [CONTAINER-EXPOSURE-EVALUATION.md](#).

Container-specific essentials (all detailed in [../docker/README.md](#)):

- **Two variants:** slim default (core + SQLite) and `-sqlserver` (adds the OS-level MS ODBC Driver 18 for the SQL Server store / `db_lookup`). Non-root uid 10001. Read-only root fs. Per-profile hash-locked deps.
- **Config is executed code:** Set mount ownership to uid 10001. Remove group/world write permission. For Kubernetes, include configuration in a derived image (`FROM messagefoundry; COPY --chown=10001:10001 config /config`).
- **Store volume must persist** (named volume / PVC, never the ephemeral layer) or the at-least-once invariant is void across a restart. Enable the at-rest cipher (`MEFOR_STORE_ENCRYPTION_KEY + MEFOR_STORE_REQUIRE_ENCRYPTION=true`).
- **Exposure:** Topology A (in-process TLS, the default) or Topology B (reverse-proxy / same-pod sidecar). A published port requires an off-loopback container bind. The bind guard therefore requires TLS, as on a physical host. Every listen type is guarded (MLLP, HTTP, the DICOM SCP, raw TCP/X12). Raw TCP/X12 have no TLS to enable, so publishing those ports means firewalling/segmenting them instead.
- **Signals:** PID 1 is `tini`. `SIGTERM` → graceful `engine.stop()`. Allow a stop grace of $\geq 30s$.

Trust boundary — inside your organization's private network

Deploy MessageFoundry inside one healthcare organization's private, trusted network, either on premises or in its private cloud. Keep it behind firewall, segmentation, and VPN/NAC controls. **Never place it directly on the public internet**. Record this trust boundary in your deployment runbook. The controls below depend on it.

The private-network boundary and the engine's bind address serve different purposes. These three areas have different exposure needs:

Plane	What it is	Where it binds	Posture
Management	web console (<code>/ui</code>) / IDE → engine API	loopback by default (or a restricted management subnet)	required auth + RBAC + full audit. Smallest surface — keep it off general-user VLANs
Data	inbound feeds you <i>receive</i> (MLLP, TCP/X12, DB-poll)	the internal network interface — feeds come from other systems on your LAN, not <code>127.0.0.1</code>	TLS on the wire (enable MLLP-over-TLS) + the <code>[egress]</code> /ingress allow-lists + your network segmentation. PHI must not cross the LAN in cleartext
Inbound web service	a partner <i>calls into</i> MEFOR (<code>Http()</code> source)	its own connector-owned socket	built (ADR 0023) — per-connection TLS + opt-in mTLS + IP allow-list, no bearer/basic partner auth . See the caveat below

The **management plane** is what you keep most contained. Data connections receive network traffic, such as EHR MLLP feeds. MLLP-over-TLS and the bind guard protect those connections. With TLS enabled on the data plane, PHI never crosses the LAN in cleartext.

Off-loopback security controls — delegate to your environment (and write it down)

Your organization's infrastructure can supply controls needed for off-loopback access within the private network. Document which controls it supplies. Also enable the required engine controls. This is an accepted way to meet the deployment-conditional OWASP ASVS items (tracked in the ASVS L3 remediation plan, an internal security-posture document not published in this repository):

Control (ASVS)	Delegate to your environment	Or build into the engine
Transport encryption (12.x)	— <i>enable</i> the shipped native API/WSS TLS + MLLP-over-TLS	already built (Gate #4)
MFA / multi-layer admin (6.3.3 / 8.4.2)	your directory (AD / Entra) — healthcare orgs are now <i>required</i> to enforce MFA there. MEFOR authenticates against it (see note below)	Native RFC 6238 TOTP MFA defaults on for local accounts (ADR 0002 WP-14). The settings are <code>[security].require_mfa = true</code> and <code>require_mfa_scope = "every_local_account"</code> , with the re-authentication gate. AD/Entra MFA stays delegated
TLS client-cert / mTLS (12.3.5)	your PKI . MF's API mTLS is built (<code>tls_client_ca_file</code> , opt-in)	enable mTLS + a console client cert
Certificate revocation (12.1.4)	your proxy / PKI (OCSP/CRL at the terminator)	document the delegation, or add OCSP/CRL to the TLS contexts
Off-box log shipping (16.4.3)	forward the audit + operational logs to your SIEM/syslog	<code>[logging].forward_*</code> sends operational logs and PHI-redacted audit rows to syslog/SIEM. Select <code>forward_protocol = "tls"</code> for RFC 5425 transport (ADR 0080, port 6514). The default is UDP. Select TLS or use a local TLS-forwarding agent.

Scoring caveat. That plan's **per-cell** scoring still reflects the pre-collapse posture columns: **ADR 0148** reduced deployment scoring to `{loopback, off-loopback}`. The owner must rescore each cell and sign again. Both actions remain pending.

Record delegated controls in your deployment runbook. Name the perimeter, identity provider, PKI, and SIEM that supply each control. Reviewers need that evidence to assess controls supplied by your environment.

Your directory (AD / Entra) acts as the identity provider and applies your MFA policy. The requirement for healthcare organizations to enforce MFA remains applicable. LDAP simple-bind checks a password but does not request a second factor.

Directory MFA therefore depends on Kerberos / Windows SSO, Conditional Access with federated SSO, or a proxy that enforces MFA. Windows SSO relies on MFA at workstation sign-in.

Local accounts use native RFC 6238 TOTP (`[security].require_mfa` , WP-14). This control is on by default for every local account. Keep it enabled. AD / SSO remains preferable when the identity provider manages MFA centrally.

Caveat — accepting inbound web-service calls

The `Http(port=...)` source accepts partner web-service requests through a separate HTTP/1.1 socket (ADR 0023). It returns `202 Accepted` after it stores the raw body in the ingress stage.

This listener has its own bind address and port. It does not inherit management API authentication. Configure its security controls separately, including inside the private network.

- **Built:** per-connection TLS (`tls = true + tls_cert_file / tls_key_file`), opt-in mTLS via `tls_ca_file`, a per-connection `source_ip_allowlist`, DoS caps (`max_connections`, `receive_timeout`, `max_body_bytes`, `max_header_bytes`), and the off-loopback exposed-gate (`check_http_tls_exposure`).
- **Not built:** The listener has no bearer/basic request authentication. Use mTLS, an IP allowlist, or a reverse proxy that authenticates requests. The synchronous downstream-reply (SOAP-envelope) path is also a defined ADR 0013 follow-on, not built: the first slice is respond-with-receipt only.

High availability — clients reach the engine through a floating VIP / L4 LB

MessageFoundry's HA model is **active-passive clustering** (N engine processes against one shared server DB. One leader runs the graph, the rest are warm standbys) — full setup in `CLUSTERING.md`. It changes the network picture in one way that belongs in your deployment plan:

- **Only the primary binds the inbound listener ports.** So senders must reach "the engine" through an operator-provided **floating VIP / L4 load balancer**, not a fixed node. Use one VIP per inbound port. Configure a TCP-connect health check for that port. Only the primary passes this check. The VIP therefore follows the primary after failover. MLLP/TCP senders see a connection drop and reconnect through the VIP (make partners reconnect-on-drop).
- The API runs on every node and accesses the shared database. An API VIP can use unauthenticated `GET /health` for liveness. To select the primary, use `GET /cluster/status (role)` or `GET /cluster/nodes (leader_node_id)`.
- **DB-tier HA is delegated to the database** (PostgreSQL streaming replication / SQL Server Always On) — MEFOR does not replicate the store itself.

MEFOR provides health-check and role endpoints but **does not include a load balancer**. Operators supply one, such as keepalived, HAProxy, F5, or a cloud NLB. Single-node deployments do not need one.

Cloud / Kubernetes HA: For multi-replica Postgres-backed HA on Kubernetes, refer to `CLOUD-DEPLOYMENT.md`. It includes a `replicas: 3` manifest and an L4-NLB-per-MLLP-port configuration. That configuration checks only the primary and excludes L7/HPA for MLLP. It also describes the hybrid edge-relay topology. For the cloud PHI / HIPAA posture (BAA, KMS, PrivateLink, region pinning), see `CLOUD-PHI-HIPAA.md` (both per ADR 0047).

Before you expose off-loopback

1. **API** — set `[security].local_access_only = false + [security].listen_address`, then either `[api].tls_cert_file + [api].tls_key_file` (in-process TLS) or `[api].tls_terminated_upstream = true + [api].trusted_proxies` (front it with a TLS terminator). Keep `[security].require_sign_in = true` (a non-loopback bind with sign-in disabled is refused, and no flag covers it). The legacy `[api].host / [auth].enabled` keys are **rejected at load** — they moved to `[security]` (ADR 0118).
2. **Web console (/ui)** — an off-loopback `/ui` **requires** in-process TLS or a declared terminator. `--allow-insecure-bind` does **not** cover the browser surface. The console is also on-by-default for *loopback* binds only: An exposed instance serves only the JSON API unless `[security].serve_web_console = true` is explicit. A declared TLS terminator also requires `[security].web_console_public_address` (the external HTTPS origin). Without it, serve refuses to start.
3. **MLLP inbound** — set `tls = true + tls_cert_file / tls_key_file` per connection. A non-loopback MLLP bind without `tls` is refused (`check_mllp_tls_exposure`). The DICOM SCP and inbound HTTP listeners have the same shape (`check_dimse_tls_exposure / check_http_tls_exposure`).
4. **Raw TCP / X12 inbound** — **no transport TLS exists** (these connectors are plaintext-only). They are, however, **exposed-gated** (since PR #558): a non-loopback raw-TCP/X12 bind is **refused at startup** (`check_tcp_tls_exposure`), parity with the MLLP/DICOM/HTTP guards. These connectors have no TLS option. Use loopback or OS-level firewall/segmentation. On default enforcing-PHI settings, `serve --allow-insecure-bind` has no effect and the bind still fails. See **no-TLS hazards**.

- Outbound connectors default to verified TLS where supported. Any `enforcement = enforce` instance rejects an off-loopback cleartext hop. This check ignores the data label, including synthetic labels. Per-connection, the only honest way across is `cleartext_accepted = true` + `cleartext_reason` (ADR 0153: warn + audit at every construction, listed by `messagefoundry check` and `GET /security/posture`). Do not set `MEFOR_ALLOW_INSECURE_TLS` in production. It does not affect cleartext-hop decisions. Where applicable, it weakens verification (see `the escape hatch`).
- Lock down egress** — populate the relevant `[egress].allowed_*` allow-lists so a transform can only send to approved destinations (see `egress allow-lists`). A PHI instance with *nothing* declared and deny-by-default off **refuses to start**.
- Off-box logs + MFA** — **both are built** and pair with off-loopback exposure: enable `[logging].forward_*` to ship logs + (PHI-redacted) audit to your SIEM (set `forward_protocol = "tls"` for the native RFC 5425 hop — the default is UDP — or front it with a local TLS agent), and leave `[security].require_mfa` on (it defaults on for every local account. AD/Entra MFA stays delegated to the IdP).

Channel × TLS posture matrix

The table uses these terms:

- Bind:** default bind or connect settings.
- TLS:** transport encryption support.
- Auth:** authentication on the channel.
- Egress gate:** the `[egress]` allowlist that controls destinations.
- Off-loopback guarded?:** whether the engine rejects a non-loopback bind without TLS.

Inbound (listeners — the engine binds a socket)

Channel	Bind default	TLS support	Auth	Ingress/egress gate	Off-loopback guarded?
Engine API (FastAPI/uvicorn)	<code>[security].local_access_only = true</code> selects <code>127.0.0.1</code>	In-process TLS uses <code>tls_cert_file / tls_key_file</code> . Upstream TLS uses <code>tls_terminated_upstream</code> and <code>trusted_proxies</code> . <code>tls_min_version</code> is ≥1.2. Optional mTLS uses <code>tls_client_ca_file</code> . HTTPS adds HSTS.	Required bearer token and session RBAC	Authentication controls access	Rejects off-loopback without TLS or a trusted terminator. Under default enforcing-PHI settings, <code>--allow-insecure-bind</code> has no effect. Rejects off-loopback when sign-in is disabled.
MLLP source	<code>[inbound].bind_host = 127.0.0.1</code>	Yes — per-connection opt-in <code>tls=true</code> + <code>tls_cert_file / tls_key_file</code> . Opt-in mTLS via <code>tls_ca_file</code> . ≥TLS 1.2. Plaintext by default	None (MLLP has no app auth)	—	Yes — non-loopback plaintext refused (<code>check_mllp_tls_exposure</code>)
HTTP source (<code>Http()</code> , ADR 0023)	<code>[inbound].bind_host = 127.0.0.1</code>	Yes — per-connection opt-in <code>tls=true</code> + <code>tls_cert_file / tls_key_file</code> . Opt-in mTLS via <code>tls_ca_file</code> . Plaintext by default	mTLS client cert only — no bearer/basic partner auth	per-connection <code>source_ip_allowlist</code>	Yes — non-loopback plaintext refused (<code>check_http_tls_exposure</code>)
DICOM C-STORE SCP (<code>DICOM()</code> , ADR 0025)	<code>[inbound].bind_host = 127.0.0.1</code>	Optional per-connection <code>tls=true</code> with certificate/key. Optional mTLS uses <code>tls_ca_file</code> . Plaintext is the default.	<code>calling_ae_allowlist</code> , <code>require_called_ae_title</code> , or mTLS. DIMSE has no separate transport authentication.	Per-connection <code>source_ip_allowlist</code>	Rejects off-loopback plaintext (<code>check_dimse_tls_exposure</code>). Also rejects an off-loopback SCP with no peer control (calling-AE allowlist, IP allowlist, or mTLS).

Channel	Bind default	TLS support	Auth	Ingress/egress gate	Off-loopback guarded?
Raw TCP source	<code>[inbound].bind_host = 127.0.0.1</code>	No — plaintext only	None	—	Yes — non-loopback plaintext refused (<code>check_tcp_tls_exposure</code> , PR #558). No TLS to enable, so keep loopback / firewall-segment / proxy-terminate
X12 source (ISA/IEA framed)	<code>[inbound].bind_host = 127.0.0.1</code>	No — plaintext only (same socket plumbing as raw TCP)	None	—	Yes — non-loopback plaintext refused (<code>check_tcp_tls_exposure</code> , PR #558). Keep loopback / firewall-segment / proxy-terminate
File source	local filesystem	n/a (no network)	n/a	—	n/a
Database poll source	Opens no listener. Connects to its own required <code>server</code> and <code>port</code> (default <code>1433</code>). It does not inherit <code>[store]</code> settings.	For <code>dialect="sqlserver"</code> , <code>encrypt</code> defaults true and <code>trust_server_certificate</code> defaults false. For <code>dialect="generic"</code> , the ODBC driver controls TLS. The engine does not enforce it.	Own <code>auth</code> , <code>username</code> , and <code>password</code>	<code>[egress].allowed_db</code>	Not applicable: outbound database connection

Outbound (the engine dials a destination)

Channel	Connect	TLS support	Auth	Egress gate
MLLP destination	dials host:port	Yes — per-connection <code>tls=true</code> . <code>tls_verify=true</code> default . Client-cert mTLS via <code>tls_cert_file</code> / <code>tls_key_file</code> + <code>tls_ca_file</code> . ≥TLS 1.2	peer HL7 ACK	<code>[egress].allowed_mllp</code>
Raw TCP destination	dials host:port	No — plaintext only	None	<code>[egress].allowed_tcp</code>
X12 destination	dials host:port	No — plaintext only	None (optional TA1)	<code>[egress].allowed_tcp</code>
REST destination	dials URL	HTTPS by default — <code>verify_tls=true</code> default (downgrade refused without the escape). Cleartext-credential <code>http</code> refused. 3xx redirects refused	optional <code>Authorization</code> (Basic/Bearer), refused over plaintext	<code>[egress].allowed_http</code>
SOAP destination	dials URL	HTTPS by default — reuses the REST client + no-redirect opener. Per-connection client-cert mTLS. ≥TLS 1.2 in the mTLS context	optional WS-Security <code>UsernameToken</code> (Nonce + Timestamp)	<code>[egress].allowed_http</code>
DATABASE destination	dials server:port	Yes — SQL Server <code>Encrypt=yes</code> default , <code>TrustServerCertificate=false</code> default (weakened only via the escape)	ODBC <code>sql</code> / <code>integrated</code> / <code>entra</code>	<code>[egress].allowed_db</code>
File destination	local filesystem	n/a (no network)	n/a	<code>[egress].allowed_file_dirs</code>
RemoteFile destination + source (SFTP / FTPS / FTP)	dials remote host	Protocol-dependent — SFTP encrypted (SSH host-key verify on by default). FTPS explicit TLS. FTP plaintext (credentials refused without the escape)	username/password or SSH key	<code>[egress].allowed_remote</code>

Above and beyond each row: One policy evaluates cleartext outbound hops for all connectors (ADR 0092, amended by ADR 0153). It allows loopback. A connection with `cleartext_accepted` and `cleartext_reason` generates a warning and audit record at each construction. Otherwise, it warns when `[security].enforcement` is not `enforce` . It rejects all remaining cleartext hops. It no longer reads the instance's data label, and `MEFOR_ALLOW_INSECURE_TLS` no longer reaches it at all.

Internal

Channel	Transport	TLS
Inter-node cluster coordination (active-passive HA / Track B)	Uses the shared [store] connection, without a separate node socket. Cluster tables hold leadership leases, heartbeats (last_seen), and configuration versions.	Uses database TLS. PostgreSQL uses DbCoordinator with asyncpg. SQL Server uses SqlServerCoordinator with aiodebc. Encrypt the store connection to encrypt cluster traffic.
Store DB connection (PostgreSQL / SQL Server)	asyncpg / aiodebc pool	Yes — [store].encrypt (default true) + [store].trust_server_certificate (default false). Weakened only via the escape

No-TLS channels — hazards

These channels have **no transport encryption at all** — there is no per-connection `tls` option as there is for MLLP:

- **Raw TCP source/destination** — plaintext, arbitrary framing.
- **X12 source/destination** — plaintext ISA/IEA-framed EDI interchanges.
- **Plain FTP** (RemoteFile `protocol=ftp`, as opposed to SFTP/FTPS) — cleartext protocol. Credentials and file contents cross the wire in the clear (the connector refuses credentials over plain FTP unless the escape is set).

Deployment requirement: run these on **loopback only**, or behind a **TLS-terminating proxy / on a trusted, isolated network segment**. If PHI flows over one of them off-host without that protection, it is exposed in cleartext. Startup rejects a non-loopback raw-TCP/X12 bind (`check_tcp_tls_exposure`, PR #558). These connectors have no TLS option. Use loopback or OS-level firewall/segmentation (`serve --allow-insecure-bind` has no effect under default enforcing-PHI settings). Choosing one (and keeping PHI off the cleartext wire) is the **operator responsibility**. On the **outbound** side these are cleartext *hops*, so they are governed by the hop authority instead: off-loopback they **refuse** on an enforcing instance unless the connection declares `cleartext_accepted + cleartext_reason`. Raw TCP and X12 require this declaration permanently because they have no `tls = true` option. See [ADR 0153](#), decision 4. TLS support remains BACKLOG #311. Credentialed plain FTP is refused outright on an enforcing PHI instance (it puts the credential itself on the wire).

The MEFOR_ALLOW_INSECURE_TLS escape hatch

Several connectors **fail closed** on a weakened-TLS or cleartext-credential configuration unless the environment variable `MEFOR_ALLOW_INSECURE_TLS` is set. It exists for **dev / trusted-lab** use only. With it set, these otherwise-refused settings become permitted (each logs a loud warning):

- REST/SOAP `verify_tls = false`. *(Clamped.)*
- MLLP outbound `tls_verify = false`. FTPS `tls_verify = false`. *(Clamped.)*
- DATABASE destination / store: `Encrypt=false` or `TrustServerCertificate=true` (SQL Server), `[store].trust_server_certificate=true / [store].encrypt=false`. *(Clamped.)*
- Plain-FTP credentials. *(Clamped.)*
- RemoteFile SFTP: accepting an unknown host key. *(Not clamped — the raw escape still applies.)*
- Cleartext SMTP submission on a **Direct** (S/MIME) destination. *(Not clamped. AUTH credentials over cleartext stay refused outright either way.)*
- The non-connection cells that have nowhere to carry a per-hop declaration: Restrictions apply to the `[logging]` syslog/SIEM forwarder and API PHI-read serve hop. LDAPS, webhook alerts, and the AI-broker endpoint still honor the raw exception variable.

The variable has two limits. [ADR 0153](#) removed it from the cleartext-hop decision. That decision also ignores the instance data label. Cleartext HTTP credentials, MLLP, DICOM, DICOMweb, and HTTP-family connections require loopback or `cleartext_accepted` plus `cleartext_reason`. The declaration produces a warning and audit record.

The engine also accepts `tls_hop_attested` as a claim that other controls secure the hop. That field has no connection factory parameter or `connections.toml` key. It is therefore unavailable through configuration, despite refusal messages that mention it.

Where `MEFOR_ALLOW_INSECURE_TLS` still applies, [ADR 0092](#) decision 2 and [ADR 0148](#) restrict most exceptions. It cannot weaken a hop under `[security].enforcement = enforce`. For MLLP, FTPS, plain FTP, and store TLS, that restriction also requires the PHI data class. Both settings are defaults. Items marked *not clamped* still honor the raw variable.

Never set `MEFOR_ALLOW_INSECURE_TLS` in production. It weakens the remaining verification checks that honor the variable.

Egress allow-lists

Outbound destinations are confined by per-protocol allow-lists in `[egress]` (`config/settings.py`). An **empty** list means unrestricted only while deny-by-default is off — which, on a PHI instance, it is not (see below). Once a list is **populated, it is fail-closed**: If a destination does not resolve to an allowed `host:port`, configuration fails at load, reload, or startup. Validation follows `env()` substitution, so it checks the resolved address.

Setting	Confines
<code>[egress].allowed_mllp</code>	MLLP destinations
<code>[egress].allowed_tcp</code>	raw TCP and X12 destinations
<code>[egress].allowed_http</code>	REST, SOAP, FHIR, DICOMweb (STOW-RS) destinations, the SMART token endpoint , and the read-only <code>fhir_lookup</code>
<code>[egress].allowed_db</code>	DATABASE destination + the DB poll source
<code>[egress].allowed_remote</code>	RemoteFile SFTP/FTPS/FTP (source + destination)
<code>[egress].allowed_file_dirs</code>	File destination directories

Webhook and SMTP alert sinks use separate allowlists: `[alerts].webhook_allowed_hosts` and `[alerts].smtp_allowed_hosts`. They carry no PHI bodies. Populate these lists separately. `[egress].allowed_http` does not control webhook alerts.

For off-loopback deployment, populate each applicable allowlist. Transforms can then send only to approved addresses.

`[security].block_unlisted_outbound` enables default denial. On PHI instances, the serve gate enables it unless you set an explicit value. With default denial active, an empty `allowed_*` list rejects every destination of that type.

A PHI instance with no populated allowlist and default denial disabled has unrestricted outbound access. Under `[security].enforcement = enforce`, that instance refuses to start. Under `warn`, it logs a warning.

Bind-guard behavior (summary)

- API** (`__main__.py`): a non-loopback bind is refused unless in-process TLS is configured, or `tls_terminated_upstream` + `trusted_proxies` are set. Also refused if `[security].require_sign_in = false`, which no flag covers. Override (dev only): `serve --allow-insecure-bind` — **clamped inert on an enforcing PHI instance**, i.e. On the shipped default.
- MLLP inbound** (`pipeline/wiring_runner.py`, `check_mllp_tls_exposure`): a non-loopback MLLP source without `tls=true` raises a `WiringError` at wiring time (before the engine starts). Override (dev only): `serve --allow-insecure-bind`, under the same clamp.
- DICOM C-STORE SCP / HTTP / raw-TCP / X12 inbound** (same module): The related checks are `check_dimse_tls_exposure`, `check_http_tls_exposure`, and `check_tcp_tls_exposure` (raw-TCP and X12, PR #558). Each rejects an off-loopback bind without TLS at wiring time. Raw-TCP/X12 are plaintext-only, so for them the only passes are loopback or OS firewall/segmentation. So every inbound listen type is now exposed-gated.
- The clamp, precisely** (ADR 0148, ADR 0092 decision 2): all four inbound gates and the API gate honour `--allow-insecure-bind` only while the instance is **not** (`enforcement = enforce` **and** PHI). Both settings are defaults, so the flag has no effect. Recorded exceptions use `[security].enforcement = warn` or `[security].handles_real_patient_data = false`. The guards read `tls_hop_attested`, and refusal messages mention it. However, no connection configuration field exposes it (see **the escape hatch**).
- Browser console** (`/ui`): Off-loopback `/ui` requires in-process TLS or a declared terminator. Without one, startup refuses the console. `--allow-insecure-bind` does not override this check.

Maintenance: keep this matrix in sync with `transports/`, `config/settings.py` (`[security]/[egress]/[api]/[store]`), `config/tls_policy.py` (the hop authority), and the bind-guards. Cross-referenced from `PHI.md` §4, `CLUSTERING.md`, and ADRs 0002 / 0148 / 0153.

© 2026 **MessageFoundry, LLC** · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated September 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MessageFoundry, LLC. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.